14 Implementing an OAuth 2 authorization server

This chapter covers

- Implementing a Spring Security OAuth 2 authorization server
- Using the authorization code and client credentials grant types
- Configuring opaque and non-opaque access tokens
- Using token revocation and introspection

Chapter 13 covered OAuth 2 and OpenID Connect. We discussed the actors that play a role in a system where the authentication and authorization are based on the OAuth 2 specification. The authorization server was one of these actors. Its role is to authenticate a user and the app they use (the client), as well as issue tokens that serve as proof of authentication to access resources protected by a backend. Sometimes, the client does that on behalf of a user.

The Spring ecosystem offers a fully customizable way to implement an OAuth 2/OpenID Connect authorization server. The Spring Security authorization server is the de facto way to implement an authorization server using Spring today. In this chapter, we'll review the main capabilities offered by this framework and implement a custom authorization server. Figure 14.1 is here to remind you about the OAuth 2 actors and the authorization server role discussed in chapter 13.

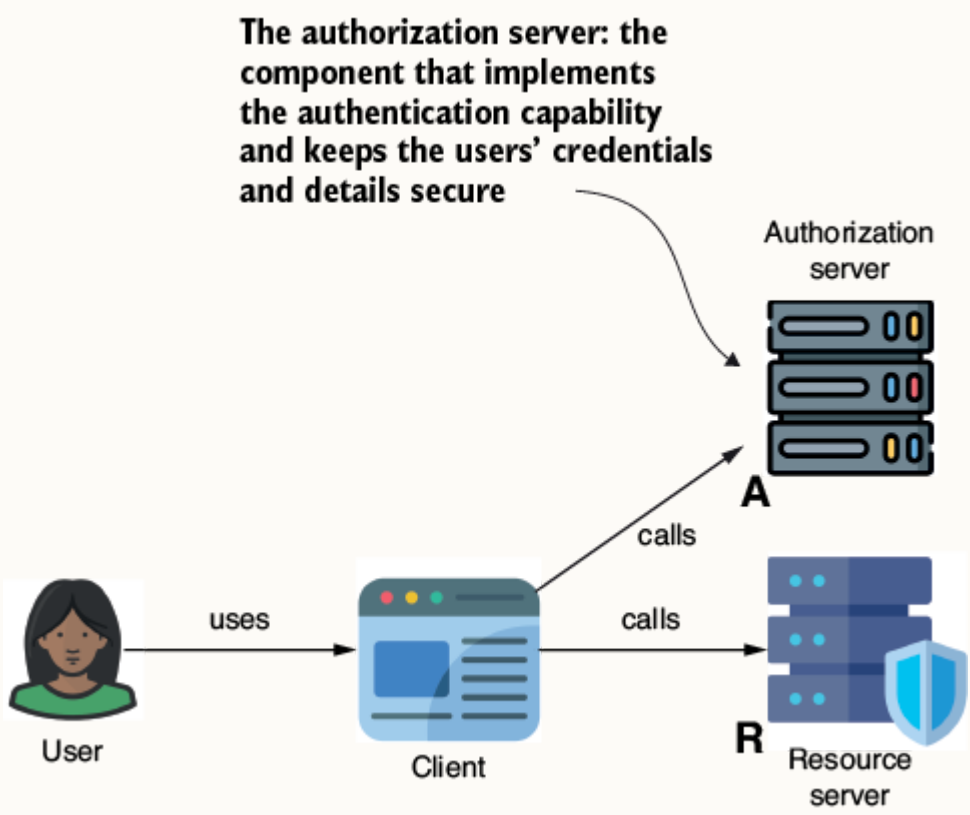


Figure 14.1 Actors in the OAuth 2 scene. The authorization server protects the user and client details and issues tokens the client can use to get authorized when calling the resource server endpoints.

We start by implementing a simple example in section 14.1, which uses the default configurations. The default configuration implies that the authorization server will issue non-opaque tokens. In section 14.2, we prove that our implementation works with the authorization code grant type, and then, in section 14.3, we demonstrate the client credentials grant type as well. In section 14.4, we continue with configuring the authorization server to work with opaque tokens and introspection. We end this chapter's discussion in section 14.5 with token revocation.

Before we start, I'd like to make you aware that the way to implement an authorization server with Spring Security is entirely different from that in the past years. In this chapter, we discuss the new approach, but you might also need to know how to implement an authorization server using an older way (for example, if you need to work on an existing app that hasn't been upgraded). In that case, I recommend reading chapter 13 of the book's first edition.

14.1 Implementing basic authentication using JSON web tokens

In this section, we employ a basic OAuth 2 authorization server using the Spring Security authorization server framework. We'll go through all the main components you need to plug into the configuration to make it work and discuss them individually. Then we'll test the app using the most essential two OAuth 2 grant types: the authorization code and

the client credentials grant type. You'll find this example implemented in project ssia-ch14-ex1.

The main components you need to set up for your authorization server to work properly are

1. *The configuration filter for protocol endpoints*—Helps you define configurations specific to the authorization server capabilities, including various customizations (which we'll discuss in section 14.3).
2. *The authentication configuration filter*—Similar to any web application secured with Spring Security, you'll use this filter to define the authentication and authorization configurations and configurations to any other security mechanisms such as cross-origin resource sharing (CORS) and cross-site request forgery (CSRF) (see chapters 2 through 10).
3. *The user details management components*—Like any authentication process implemented with Spring Security, these are established through a `UserDetailsService` bean and a `PasswordEncoder`. They work as discussed in chapters 3 and 4.
4. *The client details management*—The authorization server uses a component called `RegisteredClientRepository` to manage the client credentials and other details.
5. *The key-pairs (used to sign and validate tokens) management*—When using non-opaque tokens, the authorization server uses a private key to sign the tokens. The authorization server also offers access to a public key that the resource server can use to validate the tokens. The authorization server manages the private-public key pairs through a "key source" component.
6. *The general app settings*—A component named `AuthorizationServerSettings` helps you configure generic customizations such as the endpoints the app exposes.

Figure 14.2 illustrates the components we need to plug in and configure for a minimal authorization server app to work.

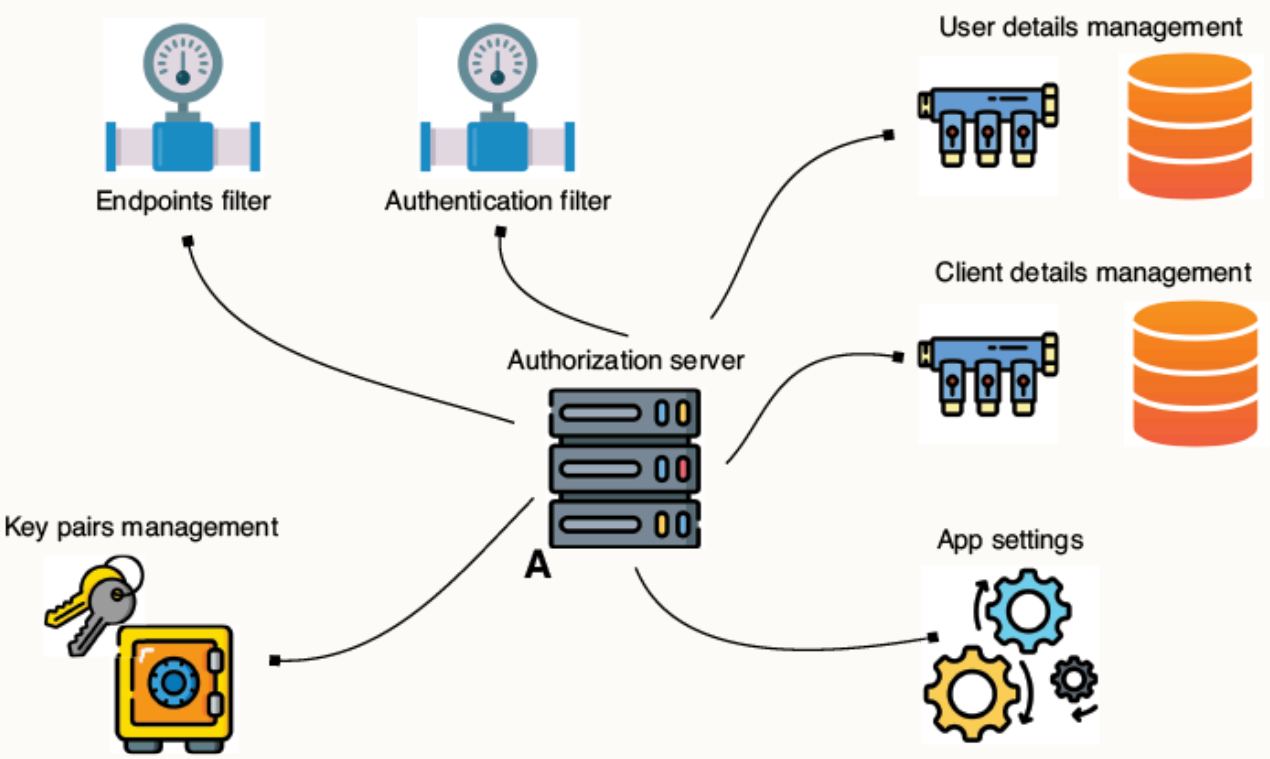


Figure 14.2 The components we need to configure and plug in for an authorization server implemented with Spring Security to work

To begin, we must add the needed dependencies to our project. In the next code snippet, you find the dependencies you need to add to your pom.xml project:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-authorization-server</artifactId>
</dependency>
```

We write the configurations in standard Spring configuration classes like in the next code snippet.

```
@Configuration
public class SecurityConfig {

}
```

Remember that like for any other Spring app, the beans can be defined in multiple configuration classes, or they can be defined using stereotype annotations (depending on

the case). If you need a refresher on managing the Spring context, I recommend the first part of *Spring Start Here* (Manning, 2021), another book I wrote.

Let's take a look at listing 14.1, which presents the configuration filter for protocol endpoints. The `applyDefaultSecurity()` method is a utility method we use to define a minimal set of configurations you can override later if needed. After calling this method, the listing shows how to enable the OpenID Connect protocol using the `oidc()` method of the `OAuth2AuthorizationServerConfigurer` configurer object.

Furthermore, the filter in listing 14.1 specifies the authentication page to which the app needs to redirect the user when asked to log in. We need this configuration because we expect to enable the authorization code grant type for our example, which implies that the users must authenticate. The default path in a Spring web app is /login, so unless we configure a custom one, we'll use this one for the authorization server configuration.

Listing 14.1 Implementing the filter for configuring protocol endpoints

```
@Bean
@Order(1)
public SecurityFilterChain asFilterChain(HttpSecurity http)
 throws Exception {

  OAuth2AuthorizationServerConfiguration                ①
    .applyDefaultSecurity(http);                        ①

  http.getConfigurer(
   OAuth2AuthorizationServerConfigurer.class)           ②
    .oidc(Customizer.withDefaults());                   ②

  http.exceptionHandling((e) ->
   e.authenticationEntryPoint(
    new LoginUrlAuthenticationEntryPoint("/login"))     ③
  );

  return http.build();
 }
```

① Calling the utility method to apply default configurations for the authorization server endpoints

② Enabling the OpenID Connect protocol

③ Specifying the authentication page for users

Listing 14.2 configures authentication and authorization. These configurations work similarly to any web app (as we discussed in chapters 2 through 10). In listing 14.2, I set up the minimum configurations:

1. Enabling the form login authentication so the app gives the user a simple login page to authenticate
2. Specifying that the app only allows access to authenticated users to any endpoints

Other configurations you could write here, besides authentication and authorization, could be for specific protection mechanisms such as CSRF (discussed in chapter 9) or CORS (discussed in chapter 10).

Observe also the `@Order` annotation I used in listings 14.1 and 14.2. This annotation is necessary because we have multiple `SecurityFilterChain` instances configured in the app context, and we need to provide the order in which they take priority in configuration.

Listing 14.2 Implementing the filter for authorization configuration

```
@Bean
@Order(2)                                    ①
public SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http)
  throws Exception {

  http.formLogin(Customizer.withDefaults());          ②

  http.authorizeHttpRequests(                        ③
    c -> c.anyRequest().authenticated()              ③
  );                                          ③

  return http.build();
}
```

① We set the filter to be interpreted after the protocol endpoints one.

② We enable the form login authentication method.

③ We configure all endpoints to require authentication.

If you expect clients will use the authorization server you build for grant types that imply user authentication (such as the authorization code grant type), then your server

needs to manage user details! Fortunately, to implement user details management, you can use the same approach you learned in chapters 3 and 4. All you need is a `UserDetailsService` and a `PasswordEncoder` implementation.

Listing 14.3 presents a definition for these two components. In this example, we use an in-memory implementation for the `UserDetailsService`, but remember that you learned how to write a custom implementation for it in chapter 3. In most cases, as you do for other web apps, you'll keep such details stored in a database. Therefore, you must write a custom implementation for the `UserDetailsService` contract.

Also, remember that in chapter 4, we discussed that the `NoOpPasswordEncoder` is something you should only use with learning samples. The `NoOpPasswordEncoder` doesn't transform the passwords in any way, leaving them in clear text and at the disposal of anyone who can access them, which is not good. You should always use a password encoder with a strong hash function such as BCrypt.

Listing 14.3 Defining the user details management

```
@Bean
public UserDetailsService userDetailsService() {
  UserDetails userDetails = User.withUsername("bill")
        .password("password")
        .roles("USER")
        .build();

  return new InMemoryUserDetailsManager(userDetails);
}

@Bean
public PasswordEncoder passwordEncoder() {
  return NoOpPasswordEncoder.getInstance();
}
```

The authorization server needs a `RegisteredClientRepository` component to manage client details. The `RegisteredClientRepository` interface works similarly to the `UserDetailsService` one, but it's designed to retrieve client details. Similarly, the framework provides the `RegisteredClient` object, whose purpose is to describe a client app the authorization server knows.

To make an analogy to what you learned in chapters 3 and 4, `RegisteredClient` is for clients what `UserDetails` is for users. Similarly, `RegisteredClientRepository` works for

client details the way a `UserDetailsService` works for user details (figure 14.3).

In this example, we'll use an in-memory implementation to allow you to focus on the overall implementation of the authorization server. Still, in a real-world app, you'd most likely need to provide an implementation to this interface to grab the data from a database. For this to work, you implement the `RegisteredClientRepository` interface similarly to how you implemented the `UserDetailsService` interface in chapter 3.

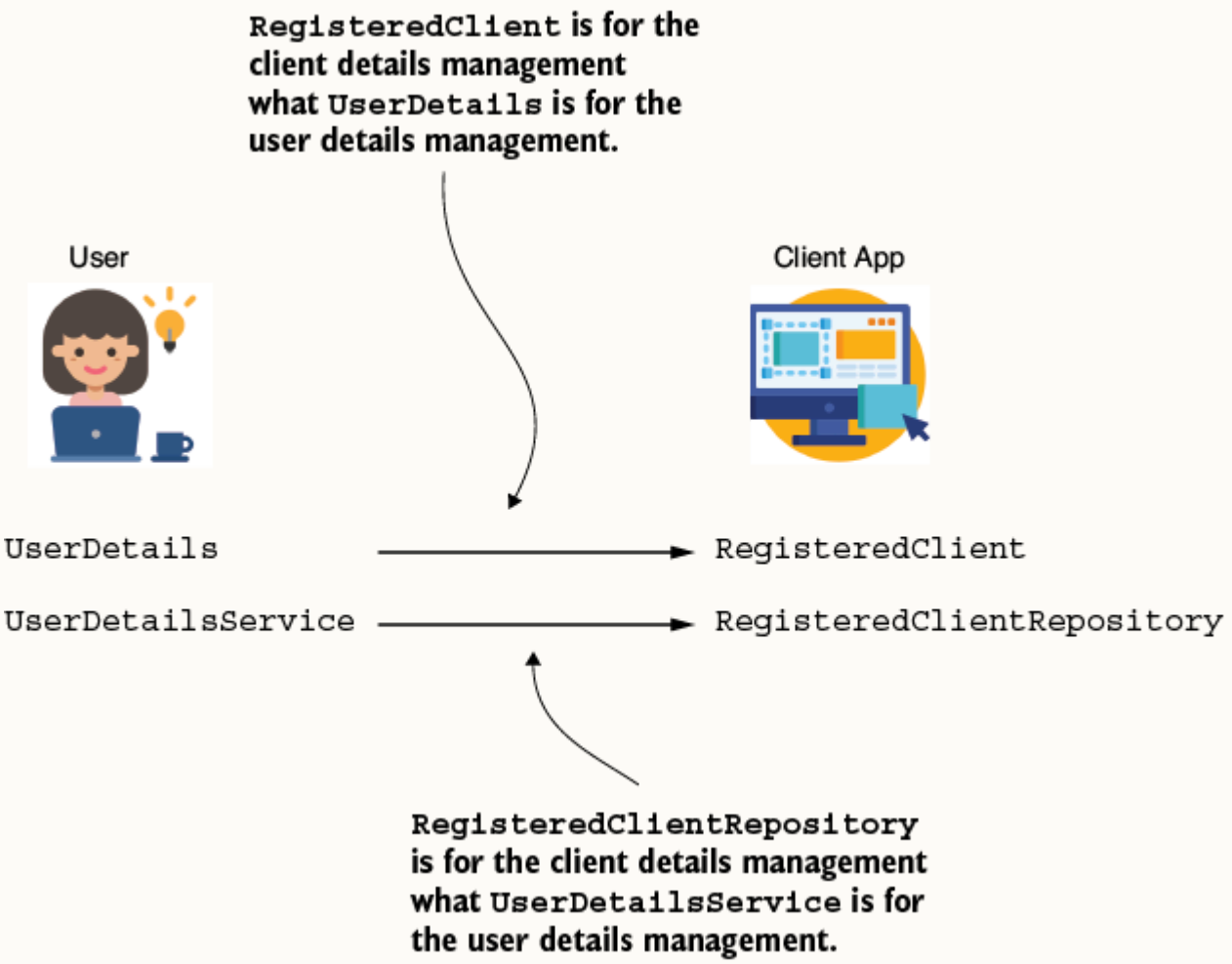


Figure 14.3 To manage client details, we use a `RegisteredClientRepository` implementation. The `RegisteredClientRepository` uses `RegisteredClient` objects to represent client details.

The next listing shows the definition of the in-memory `RegisteredClientRepository` bean. The method creates one `RegisteredClient` instance with the required details and stores it in-memory to be used during authentication by the authorization server.

Listing 14.4 Implementing client details management

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  RegisteredClient registeredClient =                    ①
    RegisteredClient                                     ①
      .withId(UUID.randomUUID().toString())              ①
```

```
        .clientId("client")                          ①
        .clientSecret("secret")                      ①
        .clientAuthenticationMethod(
          ClientAuthenticationMethod.CLIENT_SECRET_BASIC)   ①
        .authorizationGrantType(                     ①
          AuthorizationGrantType.AUTHORIZATION_CODE)       ①
        .redirectUri("https://www.manning.com/authorized")  ①
        .scope(OidcScopes.OPENID)                    ①
        .build();                                    ①

    return new InMemoryRegisteredClientRepository
      ➥(registeredClient);                          ②
  }
```

① Creating a RegisteredClient instance

② Adding it to be managed by the in-memory RegisteredClientRepository implementation

The details we specified when creating the `RegisteredClient` instance are the following:

- *A unique internal ID*—Value that uniquely identifies the client and only has a purpose in the internal app processes.
- *A client ID*—An external client identifier, similar to what a username is for the user.
- *A client secret*—Similar to what a password is for a user.
- *The client authentication method*—Tells how the authorization server expects the client to authenticate when sending requests for access tokens.
- *Authorization grant type*—A grant type allowed by the authorization server for this client. A client might use multiple grant types.
- *Redirect URI*—One of the URI addresses the authorization server allows the client to request a redirect for providing the authorization code in case of the authorization code grant type.
- *A scope*—Defines a purpose for the request of an access token. The scope can be used later in authorization rules.

In this example, the client only uses the authorization code grant type. However, you can have clients that use multiple grant types. In case you want a client to be able to use multiple grant types, you need to specify them as presented in the next code snippet. The client defined here can use any grant type (authorization code, client credentials, or the refresh token):

```
RegisteredClient registeredClient =
    RegisteredClient
        .withId(UUID.randomUUID().toString())
        .clientId("client")
        .clientSecret("secret")
        .clientAuthenticationMethod(
          ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(
          AuthorizationGrantType.AUTHORIZATION_CODE)
        .authorizationGrantType(
          AuthorizationGrantType.CLIENT_CREDENTIALS)
        .authorizationGrantType(
          AuthorizationGrantType.REFRESH_TOKEN)
        .redirectUri("https://www.manning.com/authorized")
        .scope(OidcScopes.OPENID)
        .build();
```

Similarly, by repeatedly calling the `redirectUri()` method, you can specify multiple allowed redirect URIs. Similarly, a client might also have access to multiple scopes as well. In a real-world app, the app would keep all these details in a database from where your `RegisteredClientRepository` custom implementation would retrieve them.

Besides having user and client details, you must configure key pair management if the authorization server uses non-opaque tokens (discussed in chapter 13). For non-opaque tokens, the authorization server uses private keys to sign the tokens and provides the clients with public keys they can use to validate the tokens' authenticity.

The `JWKSource` is the object providing key management for the Spring Security authorization server. Listing 14.5 shows how to configure a `JWKSource` in the app's context. For this example, I create a key pair programmatically and add it to the set of keys the authorization server can use. In a real-world app, the app would read the keys from a location where they're safely stored (such as a vault configured in the environment).

Configuring an environment that perfectly resembles a real-world system would be too complex, and I prefer you focus on the authorization server implementation. However, remember that in a real app, it doesn't make sense to generate new keys every time the app restarts (like in our case). If that happens for a real app, every time a new deployment occurs, the tokens that were already issued will not work anymore (because they can't be validated anymore with the existing keys).

So, for our example, generating the keys programmatically works and will help us demonstrate how the authorization server works. In a real-world app, you must keep the keys secured somewhere and read them from the given location.

Listing 14.5 Implementing the key pair set management

```
@Bean
public JWKSource<SecurityContext> jwkSource()
 throws NoSuchAlgorithmException {

 KeyPairGenerator keyPairGenerator =                    ①
  KeyPairGenerator.getInstance("RSA");                  ①

  keyPairGenerator.initialize(2048);                    ①
  KeyPair keyPair = keyPairGenerator.generateKeyPair();  ①

  RSAPublicKey publicKey =
   (RSAPublicKey) keyPair.getPublic();                  ①
  RSAPrivateKey privateKey =
   (RSAPrivateKey) keyPair.getPrivate();                ①
                                        ①
  RSAKey rsaKey = new RSAKey.Builder(publicKey)         ①
     .privateKey(privateKey)                    ①
     .keyID(UUID.randomUUID().toString())       ①
     .build();                                  ①

  JWKSet jwkSet = new JWKSet(rsaKey);                   ②
  return new ImmutableJWKSet<>(jwkSet);                 ③
}
```

① Generating a public-private key pair programmatically using the RSA cryptographic algorithm

② Adding the key pair to the set the authorization server uses to sign the issued tokens

③ Wrapping the key set into a JWKSource implementation and returning it to be added to the Spring context

Finally, the last component we need to add to our minimal configuration is an `AuthorizationServerSettings` object (listing 14.6). This object allows you to customize all the endpoints paths that the authorization server exposes. If you create the object as shown in the next listing, the endpoints paths will get some defaults that we'll analyze later in this section.

Listing 14.6 Configuring the authorization server generic settings

```
@Bean
public AuthorizationServerSettings authorizationServerSettings() {
  return AuthorizationServerSettings.builder().build();
}
```

Now we can start the app and test if it works. In section 14.2, we'll run the authorization code flow. Then, in section 14.3, we'll test that the client credentials flow works as expected with our authorization code implementation.

14.2 Running the authorization code grant type

In this section, we test the authorization server implemented in section 14.1. We expect that by using the registered client details, we'd be able to follow the authorization code flow and get an access token. We'll follow these steps:

1. Check the endpoints that the authorization server exposes
2. Use the authorization endpoint to get an authorization code
3. Use the authorization code to get an access token

The first step is to find the endpoint paths the authorization server exposes. Because we didn't configure custom ones, we must use the defaults. But which are the defaults? You can call the OpenID configuration endpoint in the next snippet to discover these details. This request uses the HTTP GET method, and no authentication is required:

```
http://localhost:8080/.well-known/openid-configuration
```

When calling the OpenID configuration endpoint, you should get a response that looks like the one presented in the following listing.

Listing 14.7 The response of the OpenID configuration request

```
{
   "issuer": "http://localhost:8080",
   "authorization_endpoint":
   "http://localhost:8080/oauth2/authorize",                ①
   "token_endpoint": "http://localhost:8080/oauth2/token",      ②
   "token_endpoint_auth_methods_supported": [
      "client_secret_basic",
```

```
      "client_secret_post",
      "client_secret_jwt",
      "private_key_jwt"
    ],
    "jwks_uri": "http://localhost:8080/oauth2/jwks",          ③
    "userinfo_endpoint": "http://localhost:8080/userinfo",
    "response_types_supported": [
      "code"
    ],
    "grant_types_supported": [
      "authorization_code",
      "client_credentials",
      "refresh_token"
    ],
    "revocation_endpoint": "http://localhost:8080/oauth2/revoke",
    "revocation_endpoint_auth_methods_supported": [
      "client_secret_basic",
      "client_secret_post",
      "client_secret_jwt",
      "private_key_jwt"
    ],
    "introspection_endpoint":
     "http://localhost:8080/oauth2/introspect",              ④
    "introspection_endpoint_auth_methods_supported": [
      "client_secret_basic",
      "client_secret_post",
      "client_secret_jwt",
      "private_key_jwt"
    ],
    "subject_types_supported": [
      "public"
    ],
    "id_token_signing_alg_values_supported": [
      "RS256"
    ],
    "scopes_supported": [
      "openid"
    ]
  }
```


① The authorization endpoint to which a client will redirect the user to authenticate

② The token endpoint the client will call to request an access token

③ The key-set endpoint a resource server will call to get the public keys it can use to validate tokens

④ The introspection endpoint a resource server can call to validate opaque tokens

Let's take a look at figure 14.4 to recall the authorization code flow discussed in chapter 13. We'll use it now to demonstrate that the authorization server we built works fine.

Because we don't have a client for our example, we need to act like one. Now that you know the authorization endpoint, you can put it in a browser address to simulate how the client would redirect the user to it. The next snippet shows the authorization request:

```
http://localhost:8080/oauth2/authorize?
➥response_type=code&
➥client_id=client&
➥scope=openid&
➥redirect_uri=https://www.manning.com/authorized&
➥code_challenge=QYPAZ5NU8yvtlQ9erXrUYR-T5AGCjCF47vN-KsaI2A8&
➥code_challenge_method=S256
```

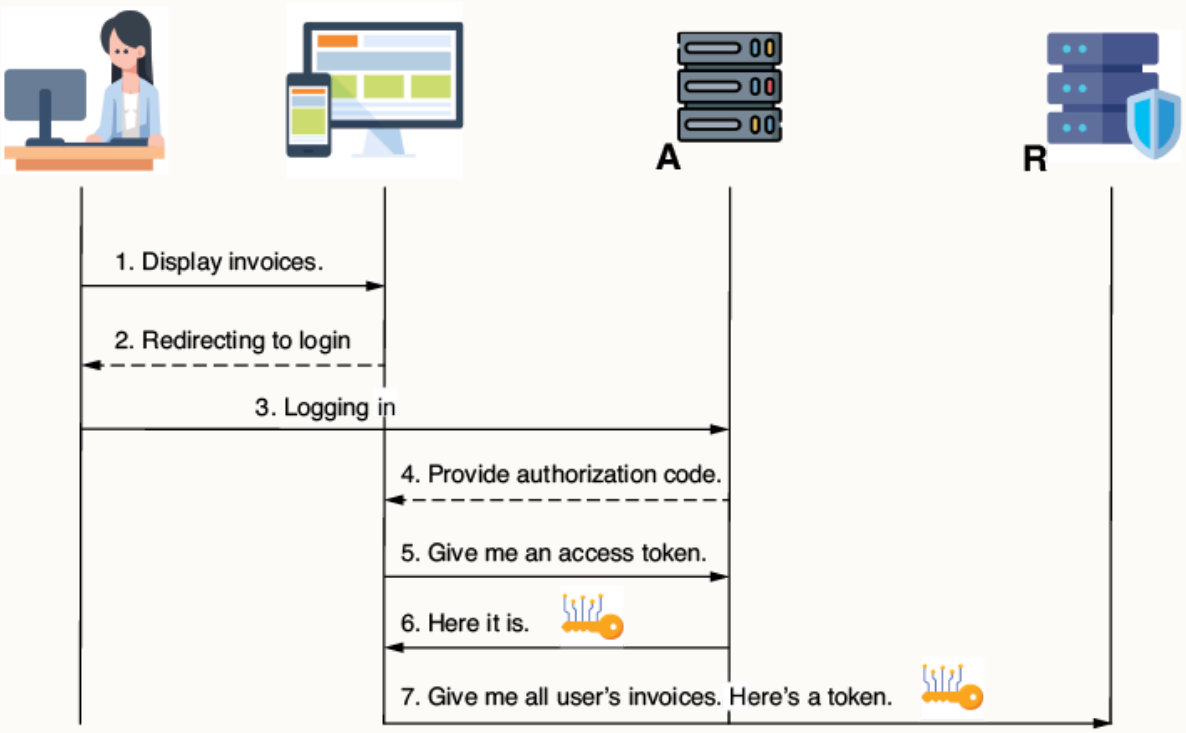


Figure 14.4 The authorization code grant type. Following successful authentication, the client receives an authorization code. This code is then used by the client to obtain an access token, which facilitates access to the resources safeguarded by the resource server.

For the authorization request, you can see I added a few parameters:

- `response_type=code` —This request parameter specifies to the authorization server that the client wants to use the authorization code grant type. Remember that a client might have configured multiple grant types. It needs to tell the authorization server which grant type it wants to use.
- `client_id=client` —The client identifier is like the "username" for the user. It uniquely identifies the client in the system.
- `scope=openid` —Specifies which scope the client wants to be granted with this authentication attempt.
- `redirect_uri=https://www.manning.com/authorized` —Specifies the URI to which the authorization server will redirect after a successful authentication. This URI must be one of those previously configured for the current client.
- `code_challenge=QYPAZ5NU8yvtlQ...` —If using the authorization code enhanced with PKCE (discussed in chapter 13), you must provide the code challenge with the authorization request. When requesting the token, the client must send the verifier pair to prove they are the same application that initially sent this request. The PKCE flow is enabled by default.
- `code_challenge_method=S256` —This request parameter specifies which hashing method has been used to create the challenge from the verifier. In this case, S256 means SHA-256 was used as a hash function.

I recommend using the authorization code grant type with PKCE, but if you really need to disable the PKCE enhancement of the flow, you can do that as presented in the next code snippet. Observe the `clientSettings()` method that takes a `ClientSettings` instance where you can specify you disable the proof key for code exchange:

```
RegisteredClient registeredClient = RegisteredClient
      .withId(UUID.randomUUID().toString())
      .clientId("client")
      // ...
      .clientSettings(ClientSettings.builder()
        .requireProofKey(false)
        .build())
      .build();
```

In this example, we demonstrate the authorization code with PKCE, which is the default and recommended way. By sending the authorization request through the browser's address bar, we simulate step 2 from figure 14.4. The authorization server will redirect us to its login page, and we can authenticate using the user name and password. This is step 3 from figure 14.4. Figure 14.5 shows the login page the authorization server presents to the user.

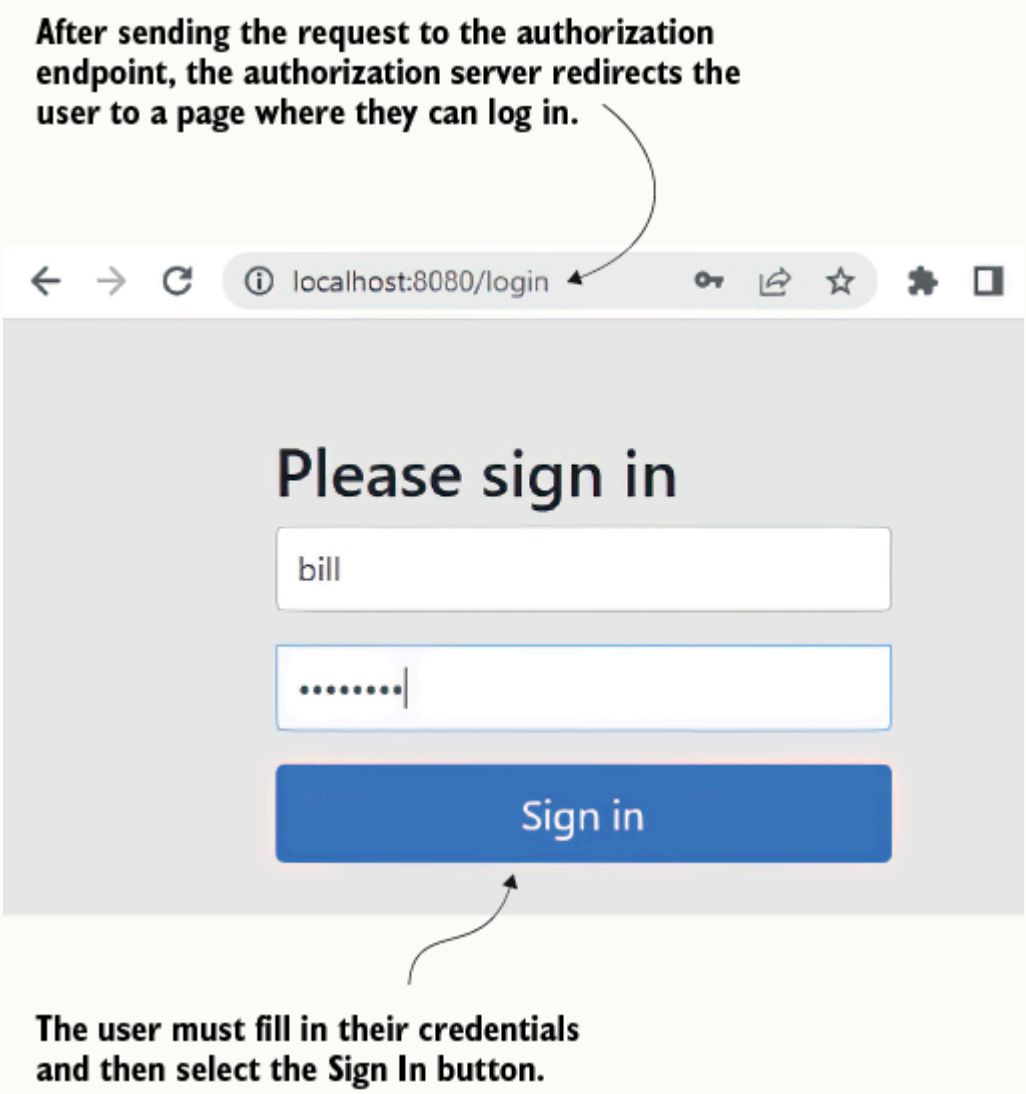


Figure 14.5 The login page that the authorization server presents to the user in response to the authorization request.

For our implementation, we only have one user (see listing 14.3). Their credentials are username `bill` and the password `password`. Once the user fills in the correct credentials and selects the Sign In button, the authorization server redirects the user to the requested redirect URI and provides an authorization code (as presented in figure 14.6; step 4 in figure 14.4).

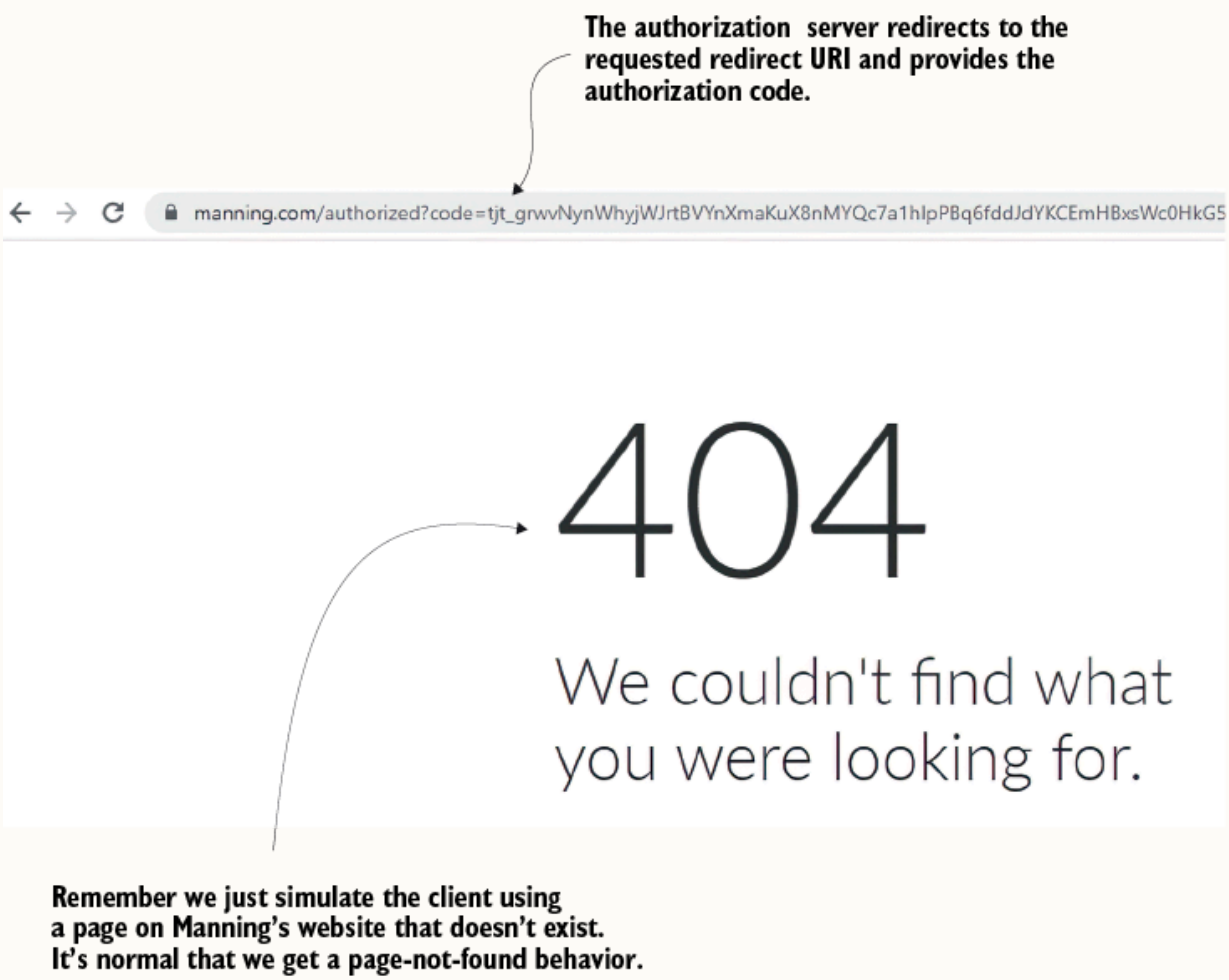


Figure 14.6 Following successful authentication, the authorization server guides the user to the specified redirect URI and issues an authorization code. The client then utilizes this code to acquire an access token.

Once the client has the authorization code, it can request an access token. The client can request the access token using the token endpoint. The next snippet shows a cURL request of a token. The request uses the HTTP POST method. Because we specified that HTTP Basic authentication is requested when registering the client, the token request needs authentication with HTTP Basic with the client ID and secret:

```
curl -X POST 'http://localhost:8080/oauth2/token?
➥client_id=client&
➥redirect_uri=https://www.manning.com/authorized&
➥grant_type=authorization_code&
➥code=ao2oz47zdM0D5gbAqtZVB...
➥code_verifier=qPsH3O6-... \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

The request parameters we used are

- `client_id=client` —Needed to identify the client

- `redirect_uri= https://www.manning.com/authorized` —The redirect URI through which the authorization server provided the authorization code after the successful user authentication
- `grant_type=authorization_code` —Shows which flow the client uses to request the access token
- `code=ao2oz47zdMOD5...` —The value of the authorization code the authorization server provided to the client
- `code_verifier=qPsH3O6-ZDD...` —The verifier based on which the challenge that the client sent at authorization was created

NOTE Pay great attention to all the details. If any of the values don't properly match what the app knows or what was sent in the authorization request, the token request won't succeed.

The next snippet shows the response body of the token request. Now the client has an access token it can use to send requests to the resource server:

```
{
   "access_token": "eyJraWQiOiI4ODlhNGFmO...",
   "scope": "openid",
   "id_token": "eyJraWQiOiI4ODlhNGFmOS1...",
   "token_type": "Bearer",
   "expires_in": 299
}
```

Because we enabled the OpenID Connect protocol, so we don't only rely on OAuth 2, an ID token is also present in the token response. If the client had been registered using the refresh token grant type, a refresh token would have also been generated and sent through the response.

Generating the code verifier and challenge

In the example we worked on in this section, I used the authorization code with PKCE. In the authorization and token requests, I used a challenge and a verifier value I had previously generated. I didn't pay too much attention to these values because they are the client's business and not something the authorization or resource server generates. In a real-world app, your JavaScript or mobile app will have to generate both values when using them in the OAuth 2 flow.

But in case you're wondering, I will explain how I generated these two values in this sidebar. You find this example in project ssia-ch14-ex2.

The code verifier is a random 32-byte piece of data. To make it easy to transfer through a HTTP request, this data needs to be Base64 encoded using an URL encoder and without padding. The next code snippet shows how to do that in Java:

```
SecureRandom secureRandom = new SecureRandom();
byte [] code = new byte[32];
secureRandom.nextBytes(code);
String codeVerifier = Base64.getUrlEncoder()
        .withoutPadding()
        .encodeToString(code);
```

Once you have the code verifier, you use a hash function to generate the challenge. The next code snippet shows how to create the challenge using the SHA-256 hash function. As with the verifier, you need to use Base64 to change the byte array into a String value, making it easier to transfer through the HTTP request:

```
MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");

byte [] digested = messageDigest.digest(verifier.getBytes());
String codeChallenge = Base64.getUrlEncoder()
        .withoutPadding()
        .encodeToString(digested);
```

Now you have a verifier and a challenge. You can use them in the authorization and token requests as discussed in this section.

14.3 Running the client credentials grant type

In this section, we'll try out the client credentials grant type using the authorization server we implemented in section 14.1. Remember that the client grant type is a flow that allows the client to get an access token without a user's authentication or consent. Preferably, you shouldn't have a client able to use both a user-dependent grant type (such as the authorization code) and a client-independent one (such as the client credentials).

As you'll learn in chapter 15, where we discuss the resource server, the authorization implementation might fail to tell the difference between an access token obtained through

the authorization code grant type and one that the client obtained through the client credential grant type. So it's best to use different registrations for such cases and preferably distinguish the token usage through different scopes.

Listing 14.8 shows a registered client able to use the client credentials grant type. Observe that I have also configured a different scope. In this case, `"CUSTOM"` is just a name I chose; you can choose any name for the scopes. The name you choose should generally make the purpose of the scope easier to understand. For example, if this app needs to use the client credentials grant type to get a token to check the resource server's liveness state, then maybe it's better to name the scope `"LIVENESS"` so it makes things obvious.

You can find the example discussed in this section in project ssia-ch14-ex3.

Listing 14.8 Configuring a registered client for the client credentials grant type

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  RegisteredClient registeredClient =
    RegisteredClient.withId(UUID.randomUUID().toString())
      .clientId("client")
      .clientSecret("secret")
      .clientAuthenticationMethod(
        ClientAuthenticationMethod.CLIENT_SECRET_BASIC)              ①
      .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
      .scope("CUSTOM")                                              ②
      .build();

    return new InMemoryRegisteredClientRepository(registeredClient);
}
```

① Allowing the registered client to use the client credentials grant type

② Configuring a scope to match the purpose for the access token request

Figure 14.7 shows the client credentials flow we discussed in chapter 13. To get an access token, the client simply sends a request and authenticates using their credentials (the client ID and secret).

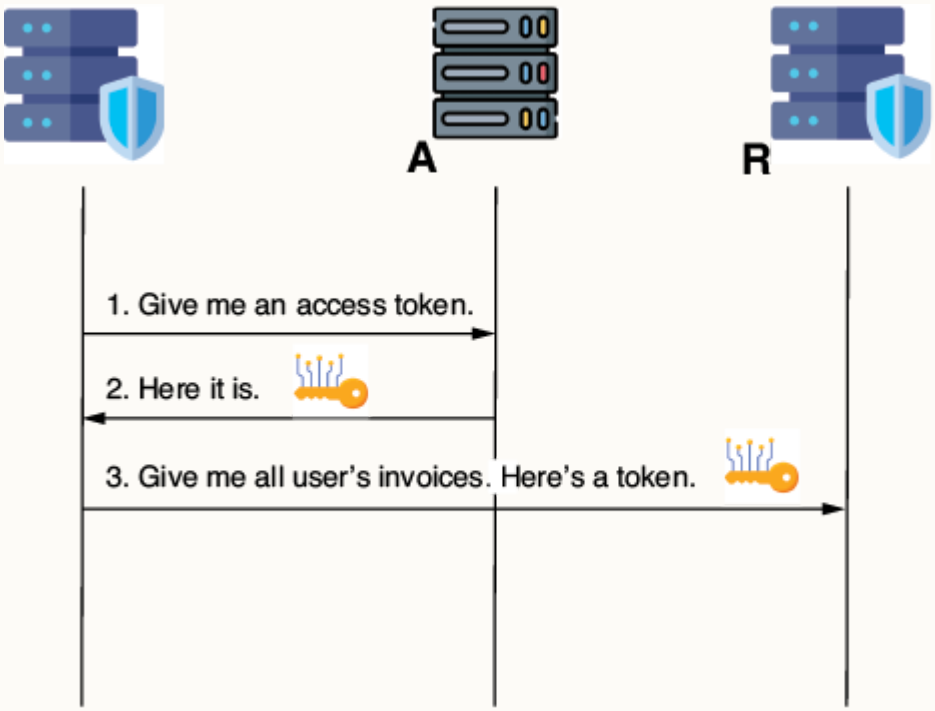


Figure 14.7 The client credentials grant type. An app can get an access token by only authenticating with its client credentials.

The following snippet shows a cURL token request. If you compare it with the request we used in section 14.2 when we ran the authorization code grant type, you'll observe that this one is simpler. The client only needs to mention they use the client credentials grant type and the scope in which they request a token. The client uses its credentials with HTTP Basic on the request to authenticate:

```
curl -X POST 'http://localhost:8080/oauth2/token?
➥grant_type=client_credentials&
➥scope=CUSTOM' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

The next snippet shows the HTTP response body containing the requested access token:

```
{
   "access_token": "eyJraWQiOiI4N2E3YjJiNS...",
   "scope": "CUSTOM",
   "token_type": "Bearer",
   "expires_in": 300
}
```

14.4 Using opaque tokens and introspection

By now in this chapter, we have demonstrated the authorization code grant type (section 14.2) and the client credentials grant type (section 14.3). With both, we managed to configure clients that can get non-opaque access tokens. However, you can also easily configure the clients to use opaque tokens. In this section, I'll show you how to configure the registered clients to get opaque tokens and how the authorization server helps with validating the opaque tokens. You find the example discussed in this section in project ssia-ch14-ex4.

Listing 14.9 shows how to configure a registered client to use opaque tokens. Remember that opaque tokens can be used with any grant type. In this section, I'll use the client credentials grant type to keep things simple and allow you to focus on the discussed subject. You can also generate opaque tokens with the authorization code grant type.

Listing 14.9 Configuring clients to use opaque tokens

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  RegisteredClient registeredClient =
    RegisteredClient.withId(UUID.randomUUID().toString())
        .clientId("client")
        .clientSecret("secret")
        .clientAuthenticationMethod(
          ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
        .tokenSettings(TokenSettings.builder()
          .accessTokenFormat(OAuth2TokenFormat.REFERENCE)       ①
          .build())
        .scope("CUSTOM")
        .build();

    return new InMemoryRegisteredClientRepository(registeredClient);
  }
```

① Configuring the client to use opaque access tokens

If you request an access token like you learned to do in section 14.3, you'll get an opaque token. This token is shorter and doesn't contain data. The next snippet is a cURL request for an access token:

```
curl -X POST 'http://localhost:8080/oauth2/token?
↪grant_type=client_credentials&
↪scope=CUSTOM' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

The following snippet shows the response similar to what you got in return when we were expecting non-opaque tokens. The only difference is the token itself, which is no longer a JWT token, but an opaque one:

```
{
   "access_token": "iED8-...",
   "scope": "CUSTOM",
   "token_type": "Bearer",
   "expires_in": 299
}
```

The next snippet shows an example of a full opaque token. Note that it is much shorter and doesn't have the same structure as a JWT (the three parts separated by dots are missing):

```
iED8-aUd5QLTfihDOTGUhKgKwzhJFzY
↪WnGdpNT2UZWO3VVDqtMONNdozq1
↪r9r7RiPOaNWgJipcEu5HecAJ75V
↪yNJyNuj-kaJvjpWL5Ns7Ndb7Uh6
↪DI6M1wMuUcUDEjJP
```

As an opaque token doesn't contain data, how can someone validate it and get more details about the client (and potentially the user) for whom the authorization server generated it? The easiest (and most used) way is to directly ask the authorization server. The authorization server exposes an endpoint where one can send a request with the token. The authorization server replies with the needed details about the token. This process is called introspection (figure 14.8).

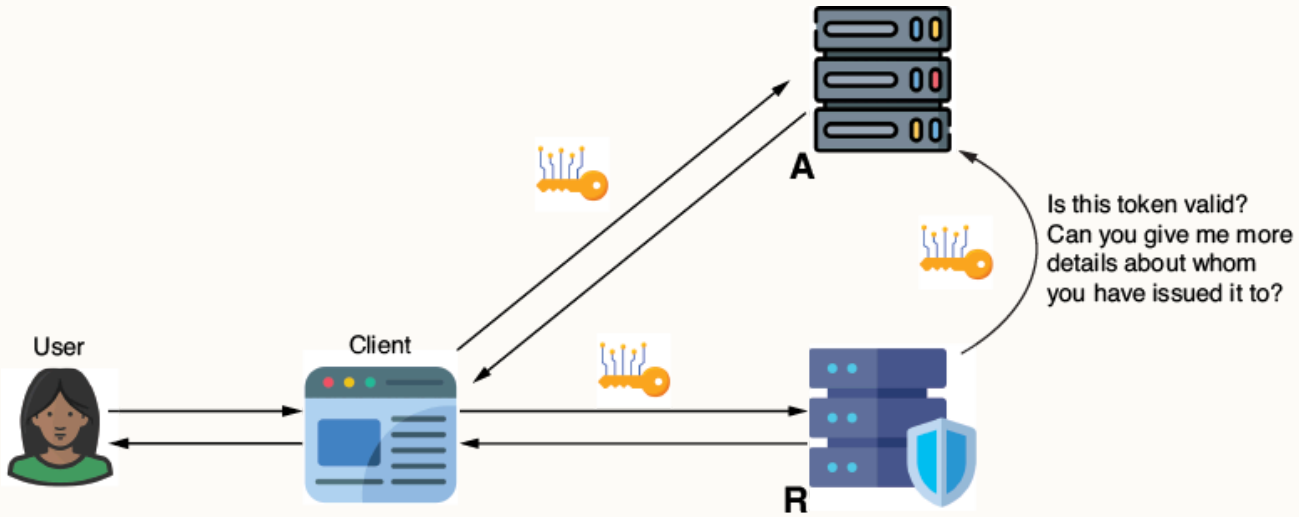


Figure 14.8 Token introspection. When using opaque tokens, the resource server needs to send requests to the authorization server to discover if the token is valid and more details about to whom it was issued.

The next snippet shows the cURL call to the introspection endpoint the authorization server exposes. The client must use HTTP Basic to authenticate using their credentials when sending the request. The client sends the token as a request parameter and receives details about the token in response:

```
curl -X POST 'http://localhost:8080/oauth2/introspect?token=iED8-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

The next snippet shows an example of a response to the introspection request for a valid token. When the token is valid, its status appears as `"active"`, and the response provides all the details the authorization server has about the token:

```
{
   "active": true,
   "sub": "client",
   "aud": [
      "client"
   ],
   "nbf": 1682941720,
   "scope": "CUSTOM",
   "iss": "http://localhost:8080",
   "exp": 1682942020,
   "iat": 1682941720,
   "jti": "ff14b844-1627-4567-8657-bba04cac0370",
   "client_id": "client",
   "token_type": "Bearer"
}
```

If the token doesn't exist or has expired, its active status is `false`, as shown in the next snippet:

```
{
   "active": false,

}
```

The default active time for a token is 300 seconds. In examples, you'll prefer to make the token life longer. Otherwise, you won't have enough time to use the token for tests, which can become frustrating. Listing 14.10 shows how to change the token time to live. I prefer to make it very large for example purposes (like 12 hours in this case), but remember to never configure it this large for a real-world app. In a real app, you'd usually go with a time to live from 10 to 30 minutes max.

Listing 14.10 Changing the access token time to live

```
RegisteredClient registeredClient = RegisteredClient
      .withId(UUID.randomUUID().toString())
      .clientId("client")
       // ...
      .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
      .tokenSettings(TokenSettings.builder()
         .accessTokenFormat(OAuth2TokenFormat.REFERENCE)
         .accessTokenTimeToLive(Duration.ofHours(12))    ①
         .build())
      .scope("CUSTOM")
      .build();
```

① Setting 12 hours as the access token time to live

14.5 Revoking tokens

Suppose you discover a token has been stolen. How could you make a token invalid for use? Token revocation is a way to invalidate a token the authorization server previously issued. Normally, an access token lifespan is short, so stealing a token still makes it difficult for one to use it. But sometimes you want to be extra cautious.

The following snippet shows a cURL command you can use to send a request to the token revocation endpoint the authorization server exposes. You can use any of the

projects we worked on in this chapter for the test. The revocation feature is active by default in a Spring Security authorization server. The request only requires the token that you want to revoke and HTTP Basic authentication with the client credentials. Once you send the request, the token cannot be used anymore:

```
curl -X POST 'http://localhost:8080/oauth2/revoke?token=N7BruErWm-44-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

If you use the introspection endpoint with a token you have revoked, you should observe that the token is no longer active after revocation (even if its time to live hasn't expired yet):

```
curl -X POST 'http://localhost:8080/oauth2/introspect?token=N7BruErWm-44-...' \
--header 'Authorization: Basic Y2xpZW50OnNlY3JldA=='
```

Using token revocation makes sense sometimes, but it's not something you'd always desire. Remember that if you want to use the revocation feature, this also implies you need to use introspection (even for non-opaque tokens) with every call to validate that the token is still active. Using introspection so often might have a big impact on the performance. You should always ask yourself: Do I really need this extra protection layer?

Remember our discussion in the first chapter. Sometimes, hiding the key under the rug is enough; other times, you need advanced, complex, and expensive alarm systems. What you use depends on what you protect.

Summary

- The Spring Security authorization server framework helps you build a custom OAuth 2/OpenID Connect authorization server from scratch.
- Since the authorization server manages the user and client details, you must implement components defining how the app collects this data:
  - To manage the user details, the authorization server needs a similar Spring Security component like any other web app: an implementation of a `UserDetailsService`.
  - To manage the client details, the authorization server provides another contract you must implement: the `RegisteredClientRepository`.
- You can register clients that use various authentication flows (grant types). Preferably, the same client shouldn't use both user-dependent (like the authorization code grant type) and user-independent (like client credentials grant type) flows.

- When using non-opaque tokens (usually JWTs), you must also configure a component to manage the key pairs the authorization server uses to sign the tokens. This component is named the `JWKSource`.
- When using opaque tokens (tokens that don't contain data), the resource server must use the introspection endpoint to verify a token's validity and collect data necessary for authorization.
- Sometimes, you'd need a way to invalidate already issued tokens. The authorization server offers the revocation endpoint for this capability. When using revocation, the resource server must always introspect the tokens (even non-opaque ones) to verify their validity.

Previous chapter
< 13 What are OAuth 2 and OpenID Connect?

Next chapter
15 Implementing an OAuth 2 resource server >